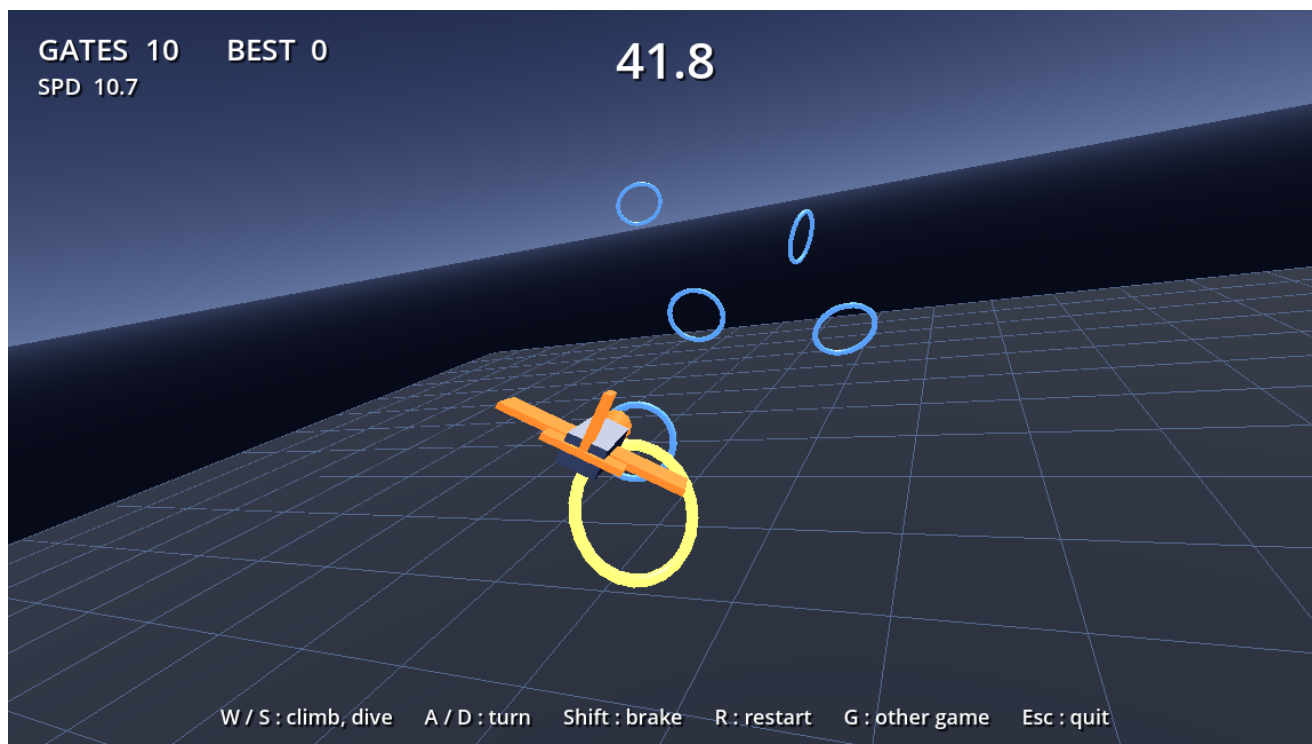


Fly By 3D

空中のリングを連続でくぐる 3D ゲーム — 人間も遊べて、AI も強化学習で上達する

Godot 4.4 + godot-rl + Stable-Baselines3 / 2026 年 8 月 19 日



黄色いリングが次の目標、青がその先。左上の GATES が得点、SPD が現在の速度。この場面では曲がりきるために減速している (10.7 m/s)。

プレイヤー	通過数 / 60 秒	中身
学習前の AI (ランダム)	0.0	何も学習していない初期方策。ただ墜落する
手書きの貪欲方策	34.4	次のリングへ機首を向け、逸れたら減速するだけ
PPO で 60 万ステップ学習した AI	39.6	本レポートで作った方策

前作 Ball Collector 3D (球を転がして的を集めるゲーム) の骨組みを流用して作った 2 本目。同じリポジトリに両方が入っていて、起動時の引数だけで切り替わる。

1 作ったもの

機体を操縦して、60 秒で空中のリングを何個くぐるかを競う 3D ゲーム。機体は常に前進していて止まれない。操作できるのは機首の向き (上下・左右) と速度だけで、リングは 1 個くぐるたびに次が前方に生成されるので、コースは事実上無限に続く。

要素	内容
ルール	60 秒タイムアタック。リングをくぐると +1 点
コース	リングは常に 6 個先まで見える。くぐった端から前方へ置き直して使い回す
操作	W / S で機首上下、A / D で左右旋回、Shift でブレーキ
速度	8 ~ 18 m/s。既定は全開で、Shift を押している間だけ減速する
失敗	地面に激突するかコース外へ出るとコースが作り直され、スタートに戻る
カメラ	機体を後方から追う。機首の向きに合わせて回る
遊び方	ブラウザ (Web ビルド) / Windows ネイティブ / コンテナ内で直接実行

このゲームの駆け引き

旋回の角速度は飛行速度によらず一定にしてある。つまり遅く飛ぶほど小回りが利く。全開のまま急なカーブに入ると曲がりきれずにリングを外し、かといって減速しっぱなしでは数が稼げない。「どこで減速するか」がそのまま上手さになる、という 1 点に絞ったゲーム設計にした。

■ 前作から引き継いだもの / 新しく作ったもの

扱い	対象
そのまま流用	学習スクリプト (train.py) / AI 観戦 (play_ai.py) / Godot 起動まわり / godot-rl プラグイン。観測と行動の次元は Godot 側から渡るので、ゲームが変わってもコードは 1 行も変えずに追従する
共用できるよう手を入れた	学習シーン (train.gd) は並べるシーンをパラメータ化。カメラ (world_view.gd) には機体追従モードを追加。起動時の分岐 (boot.gd) は --game 引数でどちらのゲームかを選ぶ形にした
新規に作成	機体 (drone) / リング (gate) / コース生成と報酬 (course) / 観測・行動・報酬の定義 (drone_ai_controller) / 人間プレイ用シーン / 基準値を測る手書き方策 (greedy_flyby.py)

前作は壊していない。ゲーム中に G キーを押すともう一方のゲームに切り替わり、学習も --game ball を付ければ前作のほうを回せる。

2 設計で一番重要だった点 — 座標系

前作と今作で決定的に違うのは、何を基準に「前」と呼ぶか。ここを外すと、人間が遊んでいるゲームと AI が学習しているゲームが別物になってしまう。

	Ball Collector 3D (前作)	Fly By 3D (今作)
制御の基準	ワールド座標系。W キーは常にワールドの -Z 方向へ転がす	機体ローカル座標系。W キーは「今向いている方向から見て上」
カメラ	追従するが回転は固定。画面の奥が常にワールド -Z	機首の向きに合わせて回る。回さないと入力と見た目がズれる
AI に渡す観測	ワールド座標のままでよい	すべて機体から見た向きに変換してから渡す

観測をローカル座標に直す理由

たとえば「リングが自分の右 10m にある」という状況は、機体が北を向いていようが南を向いていようが、取るべき行動は同じ「右に曲がる」。ところがワールド座標のまま渡すと、この 2 つは $(+10, 0, 0)$ と $(-10, 0, 0)$ というまったく違う入力になり、AI は**同じ判断を機首の向きごとに別々に覚え直す**羽目になる。機体から見た向きに直しておけば、どちらも「右に 10m」という同一の入力になり、1 回学べば全方位で使える。学習の速さがここで大きく変わる。

■ 機体の動かし方

ピッチ角とヨー角を数値として持ち、毎フレームその 2 つから姿勢を組み立て直して、機首方向へ現在の速度で進ませている。物理エンジンに任せているのは位置の積分と当たり判定だけで、「トルクをかけて姿勢が安定するのを待つ」という作りにはしていない。そのぶん挙動が素直で、人間の操作感と AI の行動空間の意味が完全に一致する。

旋回時に機体が傾く（バンクする）のは**見た目だけ**で、進行方向には影響しない。オイラー角の適用順を YXZ にしてあるので、ロールは機首方向まわりの回転になり、向きを変えずに傾きだけを表現できる。

3 強化学習で何をしているのか

使ったのは **強化学習**。「こう飛ぶのが正解」というお手本データは 1 件も与えていない。AI が自分で操縦してみ、点が入った動きの確率を上げ、墜落につながった動きの確率を下げる、という更新をひたすらくり返すだけ。

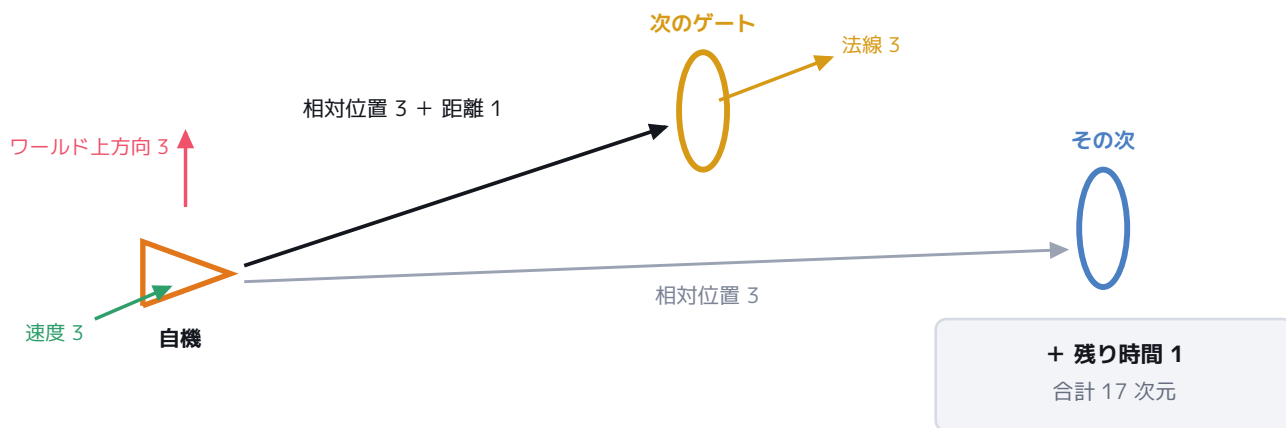


強化学習の基本ループ。この往復以外に情報は入ってこない。

■ 3-1 AI が見ているもの (観測 17 次元)

画面は見せていない。渡しているのは下の 17 個の数値だけで、すべて機体から見た向きに直し、おおむね -1 ~ +1 の範囲に収めてある。

すべて機体から見た向きに直してから渡す



観測はこの 6 種類だけ。画像も、過去の記憶も渡していない。

観測	次元	なぜ必要か
自機速度	3	曲がりきれぬかどうかは今の速度で決まる
次のリングへの相対位置	3	どちらへ向かうべきかの主信号
次のリングの法線	3	リングには表裏がある。どちら向きにくぐるべきかを示す
その次のリングへの相対位置	3	次の次が見えると、手前で減速する判断ができる
ワールド上方向 (機体から見た向き)	3	自分が今どれだけ傾いているかの手がかり
次のリングまでの距離	1	近さそのものを明示的に渡す
残り時間	1	固定長エピソードを正しく扱うために必要 (下記)

残り時間を観測に入れた理由

エピソードは 60 秒で必ず終わる。もし残り時間を見せないと、AI から見た世界は「まったく同じ状況なのに、あるときは続き、あるときは唐突に終わる」ものになり、『この状態はこの先どれくらい得点につながるか』という見積もり (価値関数) が正しく学習できない。残り時間を含めることで、いま見えている情報だけで先が決まる問題としてきちんと閉じる (マルコフ決定過程になる)。前作で効果を確認済みだったので、今作でも最初から入れた。

■ 3-2 AI が出すもの (行動 3 次元)

行動	範囲	意味
ピッチ指令	-1 ~ +1	+1 で機首上げ、-1 で機首下げ
ヨー指令	-1 ~ +1	+1 で右旋回、-1 で左旋回
スロットル指令	-1 ~ +1	-1 で 8 m/s、+1 で 18 m/s。加減速には時間がかかる

人間がキーを押したときに作られる入力とまったく同じ形式で、同じ処理に渡される。行動を決めるのは 8 物理フレームに 1 回 (action repeat = 8) なので、1 エピソード 3,600 フレームに対して AI の意思決定は 450 回。毎フレーム決めるより学習が軽くなり、しかも意思決定の粒度としては十分だった (4 回に 1 回へ細かくしても成績は変わらなかった)。

■ 3-3 報酬の設計

事象	報酬	狙い
リングをくぐった	+1.0	本来の目的そのもの
墜落・コース外	-1.0	地面や場外へ突っ込む動きを抑える
くぐらずに面を通り過ぎた	-0.2	「狙いを外した」ことを弱く伝える
次のリングに近づいた	近づいた距離 × 0.02	学習の立ち上がりを速くする

距離シェーピングが決定的だった

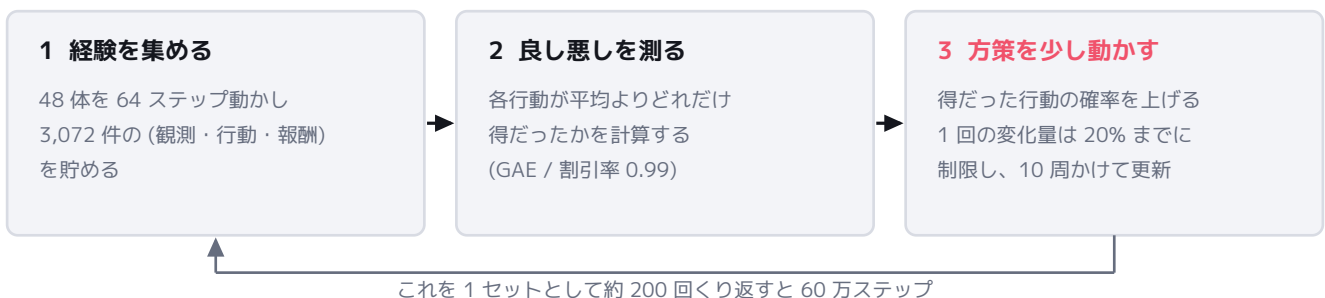
「くぐったら +1」だけだと、学習開始直後の AI は偶然リングを通り抜けるまで何の手がかりも得られない。でたらめに飛ぶ機体が直径 5m の輪を通る確率は低く、報酬がほぼ全ステップ 0 のまま時間だけが過ぎる。そこで「1 フレームで近づいた距離 $\times 0.02$ 」を毎ステップ与えた。これで AI はまず『リングの方を向いて飛ぶ』を覚え、その延長として通過にたどり着く。実測でも 3 万ステップ付近から急に伸び始めており、ここが立ち上がりの起点になっている。

シェーピングの落とし穴 — 目標が切り替わる瞬間

リングを 1 つくぐると目標が次のリングに移るので、「目標までの距離」が 0m から 20m へ一瞬で跳ね上がる。何もしないと、この 1 フレームだけ『20m 遠ざかった』という巨大な罰が入り、AI は「リングをくぐると損をする」と学んでしまう。墜落してスタートに戻された瞬間も同じ。そこで通過・墜落の直後には必ず基準距離を取り直している。前作でも踏んだ罠で、今作では最初から対策を入れた。

■ 3-4 学習アルゴリズム (PPO) が実際にやっていること

アルゴリズムは PPO (Proximal Policy Optimization)。ニューラルネットは 64×64 の 2 層 MLP で、パラメータ数は数千しかない。「観測 17 個を入ると行動 3 個が出る」だけの小さな関数で、学習とはこの関数の重みを少しずつ動かしていく作業を指す。



ポイントは 3 の「少しだけ動かす」。たまたま上手くいった経験に飛びついて方策を大きく書き換えると、それまでできていたことまで壊れて成績が崩壊する。PPO は 1 回の更新で行動確率が 20% 以上変わらないように制限をかけることで、この崩壊を防いでいる。名前の Proximal (近い) はこの制限のこと。

■ 3-5 学習を速くするための仕掛け



強化学習は「とにかく大量に試す」ことが必要なので、試行を集める速さがそのまま学習時間になる。次の 3 つで速くしている。

手段	内容	効果
コースを並べる	1 プロセスの中に独立したコースを 16 面置き、16 体を同時に飛ばす	1 回の通信で 16 体ぶんの経験
プロセスを増やす	その Godot を 3 個起動して合計 48 体	CPU コアを使い切る
物理を早送り	描画しないので実時間に縛られる理由がない。物理を 40 倍速で回す	60 秒のプレイが 1.5 秒

この構成で Godot 側は約 4,200 steps/s、PPO の勾配計算を含めた実効で約 2,950 steps/s。60 万ステップ = AI が 1,300 回ぶんの 60 秒を体験する量が、CPU だけで 約 3.4 分で終わる。

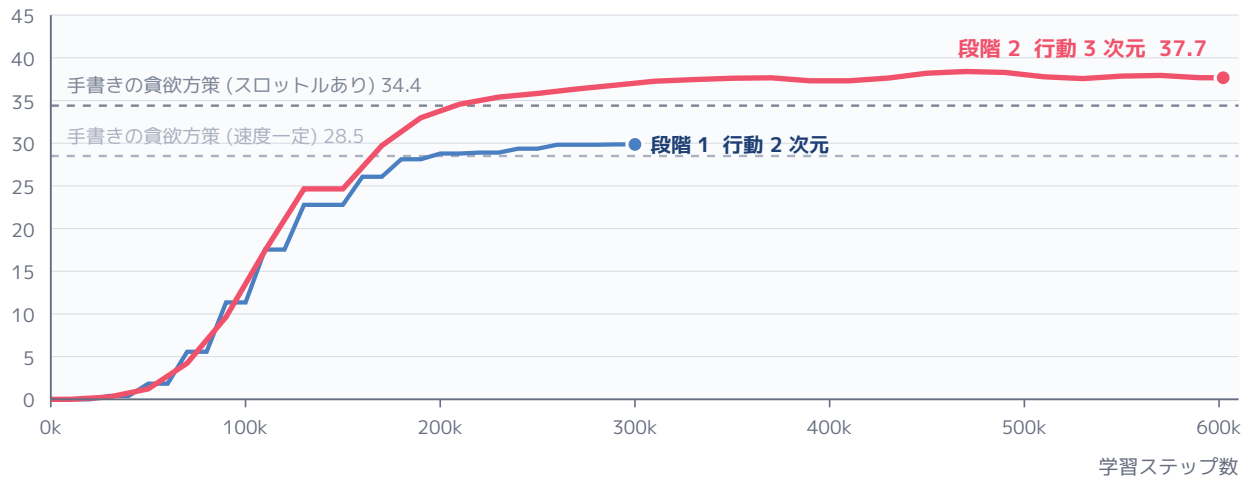
■ 3-6 学習の設定値

項目	値	選んだ理由
方策ネットワーク	64 × 64 MLP	観測が 17 次元しかないので、これで十分足りる
学習率	3e-4	PPO の標準値。触る必要が無かった
割引率 γ	0.99	60 秒先まで見通せるように長めにする
GAE λ	0.95	標準値。行動の良し悪しの見積もりを安定させる
ロールアウト長 / バッチ	64 / 256	48 体ぶんを 1 回の更新にまとめる
更新の反復	10 epoch	集めた経験を 10 周使ってから捨てる
クリップ幅	0.2	1 回の更新で方策が 20% 以上変わらないようにする
エントロピー係数	0.001	探索を少しだけ残し、序盤で動きが固まるのを防ぐ
action repeat	8	60Hz の物理に対し、意思決定は 7.5Hz
同時実行エージェント数	48	16 コース × 3 プロセス
物理の倍速	40 倍	描画しないので実時間より速く回せる

4 実際にどう上達したか

指示書の方針に従い、いきなり全部の操作を与えず 2 段階に分けて作った。段階 1 は速度一定でピッチとヨーだけ (行動 2 次元)。それが動くのを確認してから、段階 2 でスロットルを足した (行動 3 次元)。下が両方の実測カーブ。

1 エピソード (60 秒) あたりのゲート通過数



学習中の実測ログから作成。破線は比較用に書いた手書き方策の成績。

■ 何を覚えていったか (段階 2 の場合)

段階	通過数	見て分かる変化
学習前	0.0	でたらめに機首を振り、そのまま地面か場外へ突っ込む
1 万ステップ	0.0	まだ 1 個も通れない。ここで距離シェーピングが効き始める
5 万ステップ	1.2	リングの方を向いて飛ぶようになる。通るのはまだ偶然
9 万ステップ	9.7	狙って通り始める。速度は出しっぱなしで曲がりきれず外す
17 万ステップ	29.7	手書き方策 (速度一定) に並ぶ。墜落がほぼ無くなる
21 万ステップ	34.6	カーブの手前で減速するようになり、手書き方策を抜く
45 万ステップ	38.2	減速の量と再加速のタイミングが洗練される。ほぼ頭打ち

※ グラフの 37.7 は学習中 (方策からサンプリングして探索している状態) の平均。学習を止めて決定論的に動かすと **39.6 個** (最高 42 個) になる。

段階 1 と段階 2 の差が示していること

段階 1 (速度一定) では AI 31.3 に対し手書き方策 28.5 で、差はわずかだった。このゲームは「次の目標へ真っ直ぐ向かう」だけでかなりのところまで行けるので、当然ではある。ところがスロットルを与えた段階 2 では、AI 39.6 に対し手書き 34.4 と差が開いた。AI が上回っているぶんの中身は、ほぼ**速度の使い分け**。「次の次のリング」を観測に入れてあるので、まだ見えていないカーブのきつさを先読みして手前から緩めておく、という単純な追尾では出てこない振る舞いが現れた。これは報酬として明示的に教えたものではなく、報酬を最大化した結果として出てきている。

5 どう検証しながら作ったか

強化学習を含むものは「動いていないのか、下手なだけなのか」の区別が付きにくい。そこで前作で有効だった順序をそのまま踏襲し、**学習を始める前に配線ミスを潰しきることを優先した**。

順序	やったこと	そこで潰せるもの
1	ヘッドレスで読み込み、パースエラーを消す	構文・シーン構成の誤り
2	300 フレームだけ実行してエラーを消す	実行時の型・参照の誤り
3	手書きの貪欲方策でスコアを測る	観測と行動の向き (符号) の誤り。ここが最重要
4	2 ～ 3 万ステップだけ学習してみる	Python と Godot の接続、報酬の流れ
5	本番の学習を回す	—

手書きの方策を先に書く理由

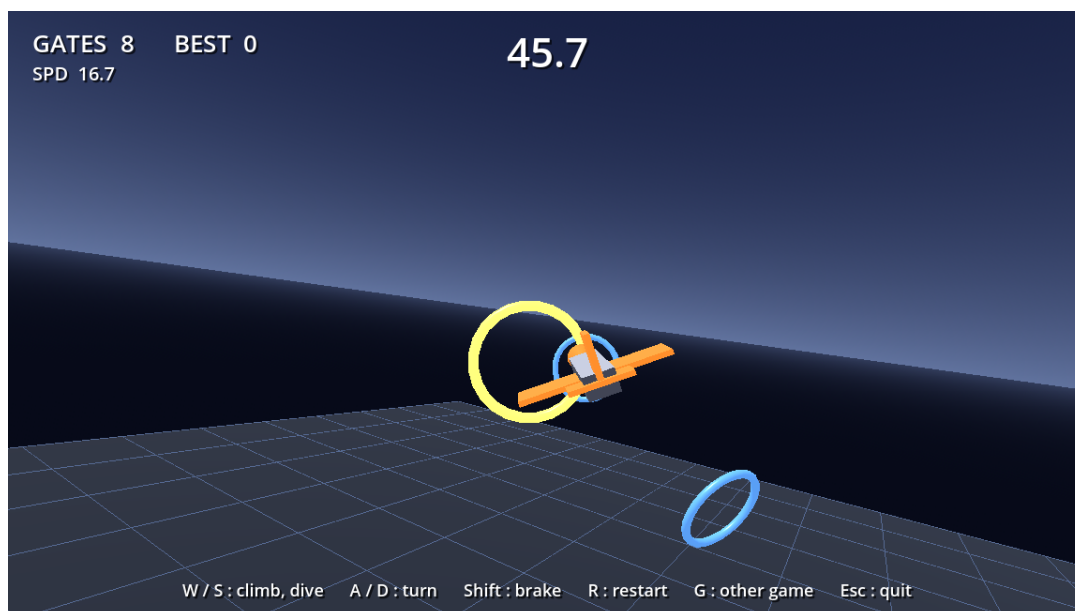
「観測から次のリングの方向を読み、そちらへ機首を向ける」だけの十数行のコードを先に書いて回す。これが 2 つの役に立つ。

1 つは**物差し**。34.4 という基準があって初めて、学習結果の 39.6 が「上手い」のか「その程度か」を判断できる。基準が無いと数字を見ても意味が分からない。

2 つめは**符号の検査**。もし観測の左右や上下を取り違えていれば、この方策はリングから遠ざかる方向へ飛ぶのでスコアがほぼ 0 になる。学習を回す前に、たった数分で向き間違いが露見する。

■ 見た目の確認

コンテナには画面が無く、X11 転送は遅いうえに操作者の画面にウィンドウが出てしまう。そこで人間プレイ用シーンを継承した使い捨てシーンを作り、貪欲方策で自動操縦しながら一定フレームごとに PNG を保存して、それを見る形にした。確認が済んだら捨てている。このレポートの写真もその方法で撮ったもの。



リングを通過する瞬間。当たり判定はリングの穴の部分だけに置いてある。

6 人間と AI が同じゲームを遊んでいること

学習した方策を別物として横に置くのではなく、人間が遊ぶときとまったく同じコードを通すように作ってある。入力元を切り替えるだけで、物理・ルール・エピソード長はすべて共通の経路を通る。



活かしている点	具体的にどうなっているか
スコアがそのまま比べられる	人間の 60 秒と AI の 60 秒が同じ長さであることを、AIController の reset_after (3,600 物理フレーム) を唯一の基準にすることで保証している。だから「AI は 39.6 個くぐれる」が、そのまま人間の目標値になる
AI のプレイを観られる	python tools/play_ai.py で、学習済みの方策が実際に飛ぶ様子をウィンドウで見られる。上達の中身 (どこで減速しているか) を目で確認できる
ゲームバランスが学習で検証された	AI が 60 秒で約 40 個くぐれる = リング間隔・旋回速度・速度域の組み合わせが「詰まらずに飛び続けられる」設定になっていることの裏付けになった
設定値の二重管理が起きない	リング間隔やコースの広さといった「人間側と AI 側の両方が知る必要のある値」はゲーム側に 1 つだけ置き、AI は観測として受け取る。バランスを変えても学習コードを直す必要がない

7 つまずいた点と、どう回避したか

問題	対処
目標が切り替わる瞬間に巨大な偽の報酬が出る	リング通過・墜落の直後は「目標までの距離」が不連続に跳ぶ。そのままだと成功したのに罰が入る。切り替えの直後に基準距離を取り直すことで解消した。しかも墜落直後は物理サーバー上の位置がまだ更新されていないため、機体の現在位置ではなく これから戻す予定の位置 から計算する必要があった
RigidBody3D を思ったとおりに動かせない	座標を直接代入しても物理サーバーに無視されることがある。姿勢と速度は物理サーバーの状態を直接書き換える形にした。また、動きが止まると物理エンジンがスリープさせてしまうため、これも無効にしている
Godot 単体で学習済みモデルを動かせない	godot-rl の ONNX 推論部は C# 実装で、この環境の Godot は非 .NET ビルドだった。そこで「Python が推論し、Godot が描画する」構成にした。将来 .NET ビルドへ移れるよう .onnx も出力してある
学習ログに「何個くぐれたか」が出せない	godot_rl 0.8.1 が、Godot から届いた付加情報を読み捨てていた。受け取り側を差し替えて通し、報酬ではなく通過数で進捗を見られるようにした。報酬は距離シェーピングを含むので直感的に読めず、これは重要だった
複数プロセスで学習を回せない	本家のラッパーが「実行ファイルが無いなら複数プロセス不可」と決め打ちしていた。自前で Godot を起動して接続する形にして 48 体並列を実現した
コンテナ内のプレイが 25fps しか出ない	描画自体 (CPU ソフトウェア描画) は 65fps 出しており、遅いのは X11 のフレーム転送だと実測で切り分けた。Web ビルドを用意し、ブラウザでのプレイを推奨する形にした

8 実測値と動かし方

■ 性能 (CPU 12 コア / RAM 7GB / GPU 無し)

項目	実測値
学習スループット (Godot 側のみ、16 コース × 3 プロセス、40 倍速)	約 4,200 steps/s
PPO の更新を含めた実効速度	約 2,950 steps/s
60 万ステップの学習時間 (CPU のみ)	約 3.4 分
メモリ (Godot ヘッドレス 1 プロセス / Python 側)	85 MB / 672 MB
描画 FPS (ローカル、1152 × 648)	65.3 fps
描画 FPS (X11 転送、1152 × 648)	12.0 fps

学習中は CPU を数分間フルに使う

Godot ヘッドレス 3 プロセス (物理 40 倍速) と PyTorch の勾配計算が同時に走るため、12 コアがほぼ 100% になり PC が熱くなる。CPU ベンチマークを回しているのと同じ負荷。温度自体は CPU 側が自動でクロックを落として守るが、負荷を下げたいときは `--parallel 1 --arenas 8 --speedup 10` を付けると 1/6 程度になる(そのぶん時間はかかる)。

■ 動かし方

やりたいこと	コマンド
ブラウザで遊ぶ	<code>python tools/serve_web.py</code> → <code>http://localhost:8000</code> を開く
AI を学習させる	<code>python tools/train.py --steps 600000</code>
AI のプレイを観る	<code>python tools/play_ai.py</code>
AI のスコアだけ測る	<code>python tools/play_ai.py --headless --episodes 10</code>
手書き方策の基準値を測る	<code>python tools/greedy_flyby.py --episodes 8</code>
前作 (Ball Collector) を動かす	上記に <code>--game ball</code> を付ける

■ 次にやるとしたら

本格的な姿勢制御 — 今の機体はピッチとヨーを直接指定する簡易モデルで、ロールは見た目だけ。角速度ではなくトルクを指令する 4 次元の行動にすると本物の飛行機に近い操縦になるが、収束には 30 分以上かかると見込まれる。

障害物と周辺認識 — プラグイン同梱のレイキャストセンサーを観測に足せば、コース上の障害物を避ける行動まで学習できる。

サバイバルモード — リングをくぐると残り時間が回復するルール。コース側の変更だけで済む。